SuryaGrid AI

Current Implementation Status and Real-Data Roadmap

Bengaluru / Karnataka / India Solar-Grid Forecasting
and DSM Multi-Agent Platform

Technical Implementation Report

Author: Lekhan HR / SuryaGrid Project

Date: 6 July 2026

Scope: Verified repository state only

Repositories: DBIT-Banglore/SuryaGrid, lekhanpro/SuryaGrid

All claims in this report are grounded in inspected files, generated artifacts, model cards, and command outputs captured on the dates shown. Items that could not be verified are marked **NOT VERIFIED**, **PENDING**, or **MISSING** rather than asserted.

Contents

Abstract

SuryaGrid AI is a multi-agent solar-grid forecasting and Deviation Settlement Mechanism (DSM) support platform focused on Bengaluru / Karnataka / India. This report documents the *verified* state of the repository as of 6 July 2026. The backend (FastAPI) exposes 54 registered endpoints across 15 routers and passes its full test suite (93 tests). A real-data ML pipeline has been executed: an hourly Bengaluru weather/solar history of 26,304 rows (2022–2024) was ingested from the Open-Meteo historical archive and cross-checked against NASA POWER (daily GHI Pearson $r = 0.87$); 344 substations were ingested from OpenStreetMap; and India national hourly demand (11,664 rows) was downloaded from Kaggle. Four models were trained and carry model cards: a solar irradiance forecaster ($R^2 = 0.956$), a cloud/irradiance-drop-risk classifier ($F_1 = 0.67$, AUC $= 0.90$), and a DSM deviation-breach classifier ($F_1 = 0.79$); a load forecaster trained on India-national data ($R^2 = 0.88$) is explicitly marked *not* production-ready due to high geographic domain shift. The reinforcement-learning policy is intentionally *not* trained (no real local load, no official tariff reward). The frontend (Next.js 14) builds successfully (14 static routes). Docker configuration exists but was *not* executed in this environment (the `docker compose` subcommand is unavailable and the daemon is unreachable). Local PV generation data, metered Karnataka load, official parsed tariff/DSM rupee rates, and substation capacity are *not* present and are tracked honestly as pending. The platform is therefore best characterised as an architecturally complete, partially data-complete prototype; it is not full production-ready.

1 Introduction

1.1 Project goal

SuryaGrid AI turns real meteorological data into a solar generation/irradiance forecast (physics via pvlib and ML via scikit-learn), compares forecast injection against a schedule, and classifies DSM deviation risk, with every reported value traceable to a source. It is organised as a set of deterministic Python agents behind a FastAPI service and a Next.js dashboard.

1.2 Why the Bengaluru / Karnataka / India context matters

Solar output, grid tariffs, deviation-settlement rules, load shape, and substation topology are all region-specific. A forecaster or DSM engine calibrated on foreign data (for example the Hawaii HI-SEAS Kaggle set) does not transfer faithfully to Bengaluru. The project therefore prioritises, in order: Bengaluru-specific, Karnataka-specific, South-India, India-wide, coordinate-based global APIs at Bengaluru's coordinates, and only then non-local data as a labelled baseline.

1.3 Data honesty taxonomy

The codebase distinguishes real data (measured/observed), estimated data (derived from real inputs via a documented formula), synthetic-augmented data (labelled, never used as production truth in real mode), and demo data. Two complementary source-tracking systems exist in the repo: a Phase 1.5 registry (`app/data_sources/source_registry.py`) and a Phase 1.7 provenance-label system (`app/ml/provenance.py`).

2 Scope of This Report

This report documents the current repository state only. Every status is derived from one or more of: directory listings, file contents, generated artifacts (parquet, pkl, JSON model cards, build/training manifests), the FastAPI route table introspected from the running application

object, the test runner, the linter, and the frontend build. Where a capability is designed but not executed or not present, it is labelled **PENDING**, **NOT VERIFIED**, or **MISSING**. No metrics, endpoints, or datasets are invented; numeric metrics are quoted verbatim from the model cards and manifests.

3 Repository Overview

The repository contains a Python/FastAPI backend, a Next.js frontend, an extensive `docs/` tree, generated ML data and models, and Docker configuration. Table 1 summarises the major components and their verified status.

Table 1: Component status (Table 1).

Component	Path	Purpose	Status	Evidence
Backend API	`backend/app`	FastAPI service, 15 routers	COMPLETE	54 routes; 93 tests
Agents	`backend/app/agents`	16 agent modules	COMPLETE	files + workflows doc
ML pipeline	`backend/app/ml`	build + train + inference	PARTIAL	artifacts generated
Data sources	`backend/app/data_sources`	providers + registry	PARTIAL	OSM/Open-Meteo wired
DSM engine	`backend/app/dsm`	rule profiles + classifier	PARTIAL	framework only (no INR)
Frontend	`frontend/`	Next.js 14 dashboard	COMPLETE	build passes, 14 routes
ML datasets	`backend/data/ml`	8 parquet + manifests	PARTIAL	files present
Trained models	`backend/models/trained`	4 pkl + rules engine	PARTIAL	4/5 trained
Model cards	`backend/models/metadata`	5 cards + run manifest	COMPLETE	all present
Tests	`backend/tests`	pytest suite	VERIFIED	93 passed
Docker	`docker-compose.yml`, Dockerfiles	4-service stack	NOT VERIFIED	not run here
Docs	`docs/`	24 markdown files	COMPLETE	inventory in Sec. 17

4 System Architecture

Figure 1 shows the high-level pipeline: real data sources are ingested and validated, materialised into ML datasets, used to train models, wrapped by agents, sequenced by an orchestrator, exposed through FastAPI, and rendered by the frontend.

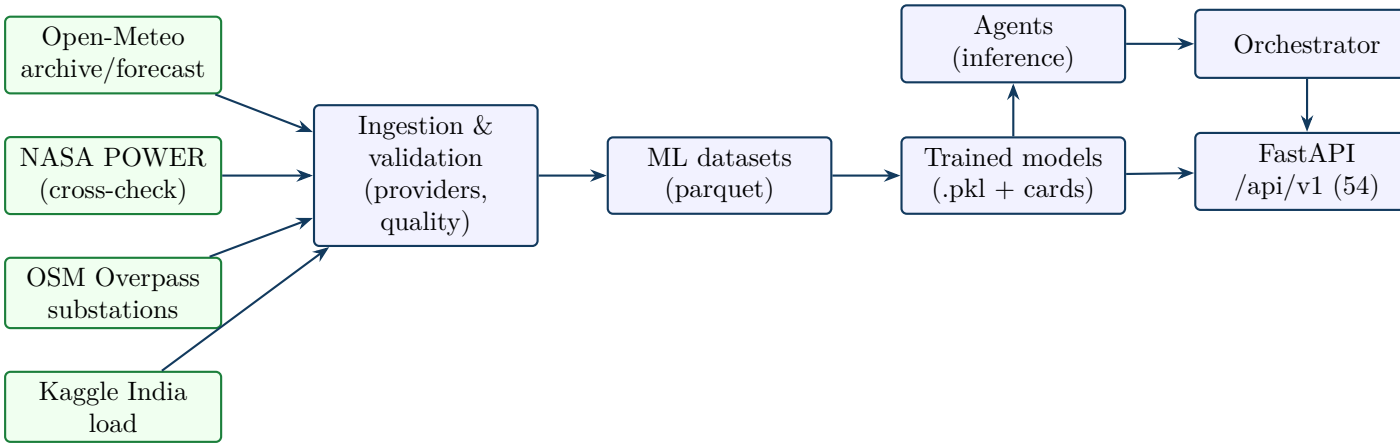

Figure 1: High-level SuryaGrid architecture (Figure 1). Green = real external sources; blue = internal stages. NASA POWER is currently wired only as a build-time cross-check, not as a live weather provider (Sec. 5).

Backend. Python 3.11/3.12, FastAPI, SQLAlchemy (SQLite default; PostgreSQL via Docker),

Redis cache (optional), pvlib + scikit-learn for the numeric path. **Frontend.** Next.js 14 / React / Tailwind, static-export capable (`frontend/out/` present). **Storage/artifacts.** ML datasets in `backend/data/ml/`, models in `backend/models/trained/`, cards in `backend/models/metadata/`.

5 Data Sources and Real-Data Status

Table 2 classifies every source found in the code, the Phase 1.5 source registry, or the Phase 1.7 build manifest. A source is only listed if it appears in the repository.

Table 2: Data source status (Table 2).

Source	Type	Geography	Implementation status	Evidence	Limitation
Open-Meteo archive	weather	coordinate (Bengaluru)	**VERIFIED** wired & used	26,304 rows; manifest	reanalysis, not on-site
Open-Meteo forecast	weather	coordinate	**VERIFIED** wired (live provider)	`source_registry.py`; tests	fair-use limits
NASA POWER	weather	coordinate (Bengaluru)	API reachable; used as cross-check; live-provider integration **PENDING**	manifest $r = 0.87$; registry "not yet wired"	not a runtime provider yet
Kaggle India load	dataset	India (national)	**VERIFIED** downloaded & used	11,664 rows parquet	non-local; domain-shift high
Kaggle HI-SEAS	dataset	Hawaii (non-local)	registered; **not** used by trained models	`source_registry.py`	PRETRAINING_ONLY only
OSM / Overpass	substation	Bengaluru/Karnataka	**VERIFIED** wired & used	344 rows parquet	voltage 41%; capacity 0%
KERC / BESCOM tariff	dsm_rule	Karnataka	framework only; rates **PENDING**	registry; verification doc	no parsed INR rates
CERC DSM 2024	dsm_rule	India	framework only; rates **PENDING**	registry; verification doc	market-linked; litigated
CEA emission/load	dataset	India	referenced in research doc; **not** wired	research doc only	no adapter/code
KPTCL-SLDC / Grid India	load	Karnataka/India	documented; **not** wired	research doc	portal/PDF, no API
Solcast	weather	coordinate	reserved slot; **not** wired	registry	requires API key
Synthetic weather	synthetic	n/a	present as labelled fallback; blocked in real mode	`synthetic_weather_provider.py`	demo only

Classification labels observed in the Phase 1.7 provenance system and cards: `REAL_COORDINATE_BASED` (weather/solar), `REAL_BENGALURU` / `REAL_KARNATAKA` (substations), `REAL_INDIA` (load), `ESTIMATED_FROM_REAL` (grid geometry, clear-sky), `NEEDS_OFFICIAL_SOURCE` (tariff/DSM rupees), `NOT_AVAILABLE` (RL environment, local PV).

6 Data Mode and Source Tracking

Table 3 records the source-tracking and data-mode mechanisms actually present.

Table 3: Data-mode and source-tracking mechanisms (Table 3a).

Mechanism	Implemented?	File / behavior	Missing work
`APP_DATA_MODE`	yes	`config.py` default `real`	UI toggle not exposed
Real-mode synthetic block	yes	`provenance.assert_real_mode_allows` raises `SyntheticFallbackError`	enforced in ML scripts; not globally in every legacy route
Phase 1.5 source registry	yes	`source_registry.py` (8 records) + `/sources` API	DB-backed re-verification
Phase 1.7 provenance labels	yes	`provenance.py`; model cards carry `source_status`, `data_mode`	unify with 1.5 registry
Model cards	yes	5 cards, required-field validation	–

The real-mode guard is exercised by the test suite (a dedicated test asserts that synthetic/demo labels raise in `real` mode). Enforcement is strongest inside the ML build/train scripts; older Phase 1 prediction routes retain a labelled synthetic weather fallback and are not governed by the same hard guard.

7 ML Dataset Creation

All datasets below were generated by `python -m app.ml.build_ml_datasets --region bengaluru --data-mode real` and are recorded in `backend/data/ml/dataset_build_manifest.json`. Row counts are quoted from that manifest and from the files on disk.

Table 4: ML dataset status (Table 3).

File (backend/data/ml/)	Exists	Geography	Source / label	Rows	Status
bengaluru_weather_solar_history.parquet	yes	Bengaluru coord	Open-Meteo+NASA / REAL_COORDINATE_BASED	26,304	**VERIFIED**
bengaluru_substations_cleaned.parquet	yes	Bengaluru/Karnataka	OSM / REAL_BENGALURU	344	**VERIFIED**
bengaluru_grid_features.parquet	yes	Bengaluru	geometry / ESTIMATED_FROM_REAL	344	**VERIFIED**
solar_agent_training.parquet	yes	Bengaluru coord	REAL_COORDINATE_BASED	26,304	**VERIFIED**
cloud_agent_training.parquet	yes	Bengaluru coord	REAL_COORDINATE_BASED	12,041	**VERIFIED**
dsm_agent_training.parquet	yes	Bengaluru coord	REAL_COORDINATE_BASED	12,031	**VERIFIED**
karnataka_or_india_load_history.parquet	yes	India national	Kaggle / REAL_INDIA	11,664	**PARTIAL** (non-local)
load_agent_training.parquet	yes	India national	REAL_INDIA	11,496	**PARTIAL** (non-local)
rl_environment_dataset.parquet	no	—	NOT_AVAILABLE	0	**MISSING**

Provenance highlights recorded in the manifest: the weather history spans 2022-01-01 to 2024-12-31 with a peak GHI of 1044 W/m2; the Open-Meteo vs NASA POWER daily GHI agreement

is Pearson $r = 0.8728$ over 1,096 days; substations have voltage on 41% of records and capacity on 0% (kept null, never invented). Each dataset row carries `source_name`, `data_geography`, `ingestion_time`, and a `quality_score` (weather mean 0.994).

8 Model Training and Model Artifacts

Metrics are quoted verbatim from the model cards in `backend/models/metadata/`. Splits are chronological 80/20.

Table 5: Model artifact status (Table 4).

Model	File / card	Metrics (held-out)	Prod?	Reason / limit
Solar irradiance	solar_forecast_model.pkl / card	R^2=0.9563, RMSE=57.51, MAE=29.12 W/m2	yes	GHI only; PV not predicted
Cloud drop-risk	cloud_risk_classifier.pkl / card	F_1=0.6707, AUC=0.9001, acc=0.866	yes	irradiance drop, not PV drop
DSM breach	dsm_classifier.pkl + dsm_rules_engine.json / card	F_1=0.7905, AUC=0.7489, acc=0.712	yes	no INR; +/-15% modelling band
Load forecast	load_forecast_model.pkl / card	R^2=0.883, RMSE=6259.5 MW	no	INSUFFICIENT_LOCAL_LOAD_DATA; domain shift HIGH
Weather correction	—	—	no	not implemented (no separate model)
RL policy	rl_policy.zip **absent** / card present	none (not trained)	no	INSUFFICIENT_REAL_ENVIRONMENT_DATA

The solar card records train/test = 21,043/5,261; cloud = 9,632/2,409; DSM = 9,624/2,407; load = 9,196/2,300. All cards report `uses_synthetic_data=false`, `synthetic_percentage=0`, and `data_mode=real`. The DSM card and rules engine assert `emits_rupee_values=false`.

9 Agent Design and Working

Per `docs/AGENT_WORKFLOWS.md`, agents are deterministic Python coordinators (no LLM in the numeric path). Sixteen agent modules exist under `backend/app/agents/`. The Phase 1.7 ML inference layer (`app/ml/agent_models.py`) additionally serves the trained models behind the `/api/v1/agents/*` endpoints. Table 6 summarises status; subsections below give per-agent detail grounded in file existence, the workflows doc, and the endpoints/cards they back.

Table 6: Agent status (Table 5).

Agent	File	Data source	Status	Limitation
API management	api_management_agent.py, api_agent.py	health/rate-limit	COMPLETE	—
Weather	live_weather_agent.py	Open-Meteo (Redis cache)	COMPLETE	synthetic fallback in legacy path

Agent	File	Data source	Status	Limitation
Solar forecast	forecast_agent.py + agent_models	Open-Meteo + solar model	PARTIAL	irradiance only
Cloud risk	agent_models (cloud_risk_classifier)	Bengaluru weather	PARTIAL	no dedicated agent class file
Location/substation	location_data_agent.py	OSM Overpass	PARTIAL	capacity/voltage often null
DSM	dsm_engine_agent.py, dsm_classifier_agent.py	profiles + DSM model	PARTIAL	framework only (no INR)
Fuzzy risk	fuzzy_risk_agent.py, risk_agent.py	deviation+weather	COMPLETE	heuristic memberships
RL / reward	reward_agent.py, app/rl/	(insufficient)	BLOCKED	not trained
Orchestrator	orchestrator_agent.py	all above	COMPLETE	–
Explanation	explanation_agent.py	forecast/DSM fields	COMPLETE	rule-based text
Persistence	persistence_agent.py	DB tables	COMPLETE	–
Feature eng.	feature_engineering_agent.py	source frames	COMPLETE	–
Source registry	source_registry_agent.py	registry	COMPLETE	static registry
Kaggle data	kaggle_data_agent.py	Kaggle API/CSV	PARTIAL	manual/API per dataset

9.1 API Manager Agent

Files: `api_management_agent.py`, `api_agent.py`. Purpose: system/provider health, rate limiting, async retry. Backs `/system/status`, `/providers/status`. Limitation: per-provider circuit breakers not implemented.

9.2 Weather Agent

File: `live_weather_agent.py` plus `data_sources/live_weather_provider.py`. Fetches Open-Meteo forecasts/history for coordinates, Redis-cached, returns provider + mode + cached flag. Handles GHI/DNI/DHI, temperature, humidity, cloud, wind, pressure, precipitation, weather code. Open-Meteo is verified/wired; NASA POWER is used only as a build-time cross-check, not as a live provider; Solcast is a reserved slot. Limitation: a labelled synthetic fallback exists in the legacy prediction path.

9.3 Solar Forecast Agent

File: `forecast_agent.py` (formula/ml/hybrid) and Phase 1.7 `agent_models.predict_solar`. Predicts **irradiance (GHI, W/m2)**, not PV AC output; no real Bengaluru/Karnataka PV generation dataset exists. Model file `solar_forecast_model.pkl` is present and production-ready for irradiance ($R^2 = 0.956$). When a user supplies capacity, PV is returned only as an explicit estimate (`estimated_output=true`, `is_actual_generation=false`) via a documented performance-ratio formula.

9.4 Cloud Risk Agent

Implemented as the Phase 1.7 classifier `cloud_risk_classifier.pkl` served by `agent_models.predict_cloud` at `/agents/cloud/risk`; there is no dedicated `cloud_agent.py` class file. Label is `irradiance_drop_risk` defined by clear-sky clearness index $k_t < 0.5$ during daylight (documented in `docs/formulas.md`). Metrics: $F_1 = 0.67$, AUC $= 0.90$. Limitation: irradiance drop, not PV-output drop.

9.5 Location / Substation Agent

File: `location_data_agent.py` plus `data_sources/substation_provider.py`. Source: OSM Overpass (`power=substation`, ODbL). Fields: id, name, lat/lon, operator, voltage_kv, capacity_mva (kept null), district, source, reliability_score, missing_fields. 344 substations ingested; capacity is unavailable from OSM and is *not* assumed.

9.6 DSM Agent

Files: `dsm_engine_agent.py`, `dsm_classifier_agent.py`, plus `app/dsm/` rule profiles and `dsm_rules_engine.json`. Produces a deviation band (WITHIN_BAND / OVER / UNDER) and a breach-risk probability; it does **not** compute rupee charges. Tariff/DSM rupee rates are `NEEDS_OFFICIAL_SOURCE` (framework verified from CERC 2024 / KERC, but exact rates market-linked and unparsed).

9.7 RL / Reward Agent

File: `reward_agent.py` and `app/rl/`. Status: experimental scaffold. No policy is trained; `rl_policy.zip` is absent and the RL card records `production_ready=false`, `reason=INSUFFICIENT_REAL_ENVIRONMENT_DATA` (missing real local load and official tariff reward). This is intentional; a policy trained on fabricated rewards is explicitly refused.

9.8 Orchestrator Agent

File: `orchestrator_agent.py`. Sequences Forecast → DSM → Risk → Explanation for an interval and assembles the combined response used by the prediction endpoints and dashboard.

10 Agent Workflow Diagrams

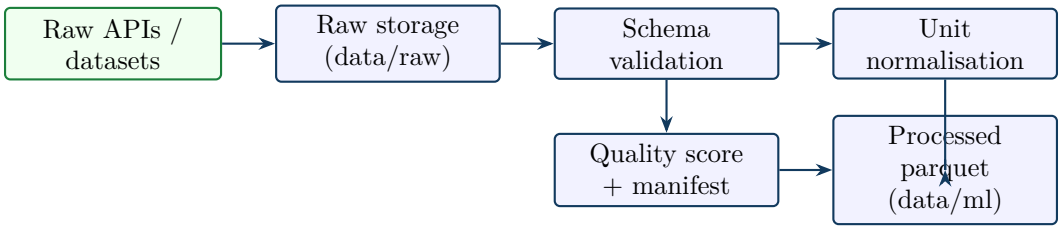

Figure 2: Data ingestion and validation workflow (Figure 2).

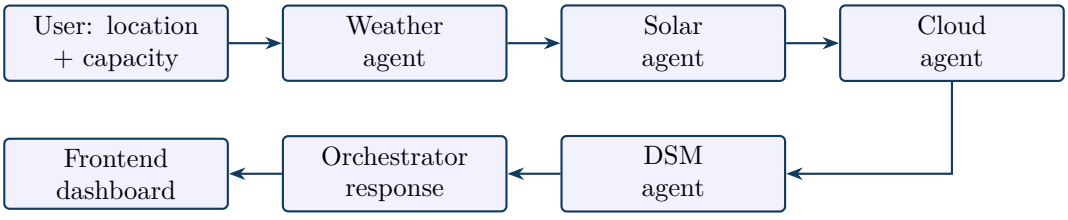

Figure 3: Forecast / orchestration workflow (Figure 3).

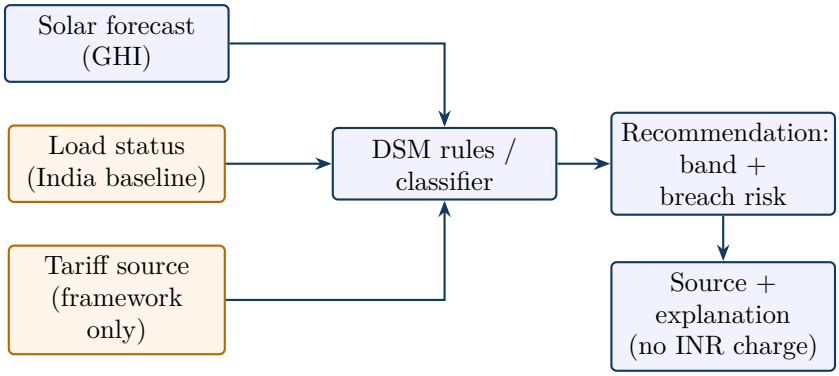

Figure 4: DSM decision workflow (Figure 4). Orange nodes denote inputs that are non-local or framework-only, so the output is a band/risk recommendation, not a rupee settlement.

11 API Layer

The application registers 54 endpoints under `/api/v1` across 15 routers (introspected from the FastAPI `app.routes` object). Table 7 lists them. The Phase 1.7 `/agents/*` endpoints return a full provenance envelope (`prediction_type`, `model_file`, `model_version`, `training_geography`, `target_geography`, `local_data_used`, `source_status`, `confidence_components`, `production_ready`, `data_mode`).

Table 7: Registered API endpoints (Table 6).

Method	Path (/api/v1)	Provenance?
GET	/health	–
GET	/system/status	–
POST	/agents/solar/forecast	yes
POST	/agents/cloud/risk	yes
POST	/agents/dsm/assess	yes
GET	/agents/status	yes
GET	/agents/data-status	yes
GET	/sources ; /sources/{id}	yes
GET	/data-sources/status	yes
POST	/dsm/check ; /dsm/advanced-check	partial
GET/POST	/dsm/rule-profiles ; /dsm/rule-profiles/{id}	yes
POST	/dsm/karnataka/{site_id}	partial
GET	/locations ; /locations/available	–
GET	/substations ; /substations/nearest/{site_id}	yes
POST	/substations/import	–
GET	/weather/{site_id} ; /weather/current/{site_id}	yes
GET	/weather/latest/{site_id} ; /weather/forecast/{site_id}	yes
POST	/weather/fetch ; GET /weather/providers/status	yes
GET	/forecast/{site_id}	yes
POST	/predict ; GET /predict/{site_id}	yes
GET	/summary/{site_id} ; /timeline/{site_id}	partial
POST	/ml/train ; /ml/predict	yes
POST	/ml/datasets/ingest-kaggle ; /ml/datasets/build-augmented	–
GET	/ml/model/status	yes
GET/POST	/sites ; /sites/{id} ; /sites/{id}/data-coverage	–
GET	/energy/{site_id}	partial

Method	Path	Provenance?
POST	/settle ; /settle/day/{site_id} ; GET /settlements/{site_id}	partial
GET/POST	/rl/rates ; /rl/runs ; /rl/train	partial
POST	/ingest/current/{site_id}	–
GET	/karnataka/regions ; /bescom/status ; POST /karnataka/seed	partial
GET	/consumption/profiles	–

12 Frontend Implementation

The frontend is Next.js 14 with 13 page routes and shared components. A production build succeeds and static-exports 14 routes (including `/_not-found`). Table 8 lists the pages.

Table 8: Frontend pages (Table 7).

Page	Path	Purpose	Size	Status
Root	app/page.tsx	landing	176 B	**COMPLETE**
Dashboard	app/dashboard	forecast + provenance panel	5.3 kB	**COMPLETE**
Data sources	app/data-sources	source/registry status	3.6 kB	**COMPLETE**
DSM	app/dsm	DSM check UI	4.3 kB	**COMPLETE**
ML	app/ml	model/metrics view	4.3 kB	**COMPLETE**
Locations	app/locations	sites/substations	4.1 kB	**COMPLETE**
System	app/system	health/status	3.6 kB	**COMPLETE**
Karnataka	app/karnataka	regional view	3.7 kB	**COMPLETE**
Predictions	app/predictions	prediction view	4.1 kB	**COMPLETE**
Timeline	app/timeline	time-series	2.2 kB	**COMPLETE**
Energy	app/energy	energy balance	7.8 kB	**COMPLETE**
Settlement	app/settlement	settlement view	3.7 kB	**COMPLETE**
RL	app/rl	RL rates/runs	4.4 kB	**COMPLETE**

Components include `ModelProvenancePanel.tsx` (renders backend data mode, geography, production readiness, and honesty warnings) and `OfflineBanner.tsx` (shows an explicit offline state instead of faking data). "Status" here means the page builds and renders; live data depends on a running backend.

13 Testing and Verification

Table 9 lists commands actually run for this report and their outcomes.

Table 9: Test / verification results (Table 9).

Command	Result		Notes
`python -m pytest tests/ -q`	**VERIFIED** passed	93	1 benign warning (CPU-count)
`ruff check app tests`	**VERIFIED** passed	all	no lint errors
`ruff format --check app tests`	**PARTIAL**		10 files would reformat (style only)
`npm run build` (frontend)	**VERIFIED** success		14 routes static-exported
FastAPI route introspection	**VERIFIED**		54 endpoints enumerated
`docker --version`	**VERIFIED**		Docker 29.6.1 installed
`docker compose version`	**MISSING**		"unknown command: docker compose"
`docker compose config`	**NOT VERIFIED**		cannot run (subcommand unavailable)
`docker info` (daemon)	**NOT VERIFIED**		daemon unreachable
Kaggle load ingestion	**VERIFIED**		11,664 rows downloaded/used
NASA POWER cross-check	**VERIFIED**		$r = 0.87$ (manifest)

14 Docker and Deployment State

`docker-compose.yml` defines four services with healthchecks and a named volume. Docker Engine is installed (v29.6.1) but the `docker compose` subcommand is unavailable in this environment and the daemon is unreachable, so the stack was **not** built or run. Deployment status is therefore **NOT VERIFIED**.

Table 10: Docker services (Table 8).

Service	Port	Purpose	Config		Verified?
postgres	5432	PostgreSQL 16 database	compose healthcheck	+	no
redis	6379	Redis 7 cache	compose healthcheck	+	no
backend	8000	FastAPI (build ./backend)	Dockerfile healthcheck	+	no
frontend	3000	Next.js (build)	Dockerfile healthcheck	+	no
pgdata	—	Postgres volume	named volume		no

Note: `.env.example` is present but was not read for this report (blocked by the workspace secret-file access policy); environment variables are inferred from `app/config.py` (e.g. `APP_DATA_MODE`, `DATABASE_URL`, `REDIS_URL`, `WEATHER_PROVIDER`).

15 Current Limitations

- **No local PV generation data.** Models predict irradiance; PV output is an explicit estimate from user capacity, never measured or directly predicted.
- **No metered Karnataka load/injection.** The only real load series is India national (Kaggle); the load model is marked not production-ready (domain shift HIGH). "Actual injection" remains a nowcast/estimate.
- **Tariff/DSM rupee rates not parsed.** CERC/KERC frameworks are verified, but exact rates are market-linked and unparsed; DSM output is framework-only (`emits_rupee_values=false`).

- **Substation capacity unavailable.** OSM provides locations and partial voltage (41%) but no capacity; substation-level DSM is not claimed.
- **Non-local baselines.** India-national load and HI-SEAS remain pretraining/ baseline only, never production truth.
- **Docker not run.** Compose subcommand unavailable and daemon unreachable here.
- **NASA POWER not a live provider.** Reachable and used as a build cross-check; runtime provider integration is pending.
- **RL not production-ready.** No policy trained; requires real local load, solar, and official tariff reward.
- **Lint format drift.** `ruff check` passes; `ruff format --check` would reformat 10 files (cosmetic).

16 What Should Be Added Next

Table 11: Prioritised roadmap (Table 10).

P	Addition	Why	Source required	Risk
1	Karnataka SLDC / Grid India load ingestion	localise load; unblock DSM/RL	KPTCL-SLDC / Grid India	portal/PDF, no API
2	Official KERC/BESCOM tariff + CERC DSM parser	real INR DSM charges	KERC/CERC orders	PDF parsing, litigated rates
3	Local PV generation ingestion	real PV target, not estimate	plant/BESCOM feed	availability
4	Substation capacity enrichment	substation-level DSM	KPTCL/BESCOM/GESCOM scanned maps	
5	NASA POWER live provider	second runtime weather source	NASA POWER API	LST alignment
6	Strict global APP_DATA_MODE enforcement	block synthetic in all routes	(code)	legacy path refactor
7	Docker full-stack verification	deployment readiness	Docker Desktop	env-specific
8	Retrain load model on local data	production-grade load	item 1	depends on item 1
9	RL environment (after 1,2,3)	dispatch optimisation	real load+tariff	only after data exists
10	Unify 1.5 registry + 1.7 provenance	single source-of-truth	(code)	low

17 Conclusion

SuryaGrid AI is an architecturally complete multi-agent platform with a verified backend (54 endpoints, 93 passing tests), a building frontend (14 routes), and an executed real-data ML pipeline for Bengaluru weather/solar, substations, and India-baseline load. Three models are production-ready for their stated (irradiance/risk/deviation-band) tasks; the load model and RL policy are honestly marked not production-ready. It is *not* full production-ready: local PV, metered local load, official tariff rupees, and Docker validation are outstanding.

Table 12: Readiness classification.

Dimension	Level	Basis
Architecture	**COMPLETE**	agents + API + frontend + ML pipeline all wired and running
Data	**PARTIAL**	real weather/solar + substations + India load; local PV/load/capacity missing
ML	**PARTIAL**	solar/cloud/DSM prod-ready; load non-local; RL not trained
DSM	**PARTIAL**	framework + classifier; official INR rates pending
Deployment	**NOT VERIFIED**	compose/Dockerfiles present; not built/run here
Production	**PENDING**	blocked on local PV, local load, official tariff, Docker validation

Appendix A: Key command outputs

```
$ python -m pytest tests/ -q
93 passed, 1 warning in ~14s

$ ruff check app tests
All checks passed!

$ npm run build   (frontend, Next.js 14)
14 routes prerendered as static content (/, /dashboard, ... /timeline)

$ docker --version
Docker version 29.6.1, build 8900f1d
$ docker compose version
docker: unknown command: docker compose        # compose plugin unavailable

Build manifest (backend/data/ml/dataset_build_manifest.json):
  weather_solar_history: 26304 rows, 2022-01-01..2024-12-31,
    NASA POWER cross-check r=0.8728 (1096 days), peak GHI 1044 W/m2
  substations: 344 (voltage 41%, capacity 0%)
  load_history: 11664 rows (REAL_INDIA)
  rl_environment_dataset: NOT_AVAILABLE
```

Appendix B: Generated artifacts

Datasets (`backend/data/ml/`): bengaluru_weather_solar_history, bengaluru_substations_cleaned, bengaluru_grid_features, solar_agent_training, cloud_agent_training, dsm_agent_training, karnataka_or_india_load_history, load_agent_training (all .parquet); dataset_build_manifest.json; tariff_dsm_rules_official_or_pending.json.
Models (`backend/models/trained/`): solar_forecast_model.pkl, cloud_risk_classifier.pkl, dsm_classifier.pkl, load_forecast_model.pkl, dsm_rules_engine.json. (`rl_policy.zip` absent by design.)
Cards (`backend/models/metadata/`): five `*_card.json` + training_run_manifest.json.

Appendix C: Documentation inventory

Present (`docs/`): AGENT_ARCHITECTURE, AGENT_WORKFLOWS, API_DESIGN, API_REFERENCE, BENGALURU_DATA_AND_ML_PIPELINE, BENGALURU_DATA_SOURCE_RESEARCH, BROWSER_AND_DATA_ACCESS_TEST, DATA_SOURCE_CATALOG, DEPLOYMENT, DOCKER_ARCHITECTURE, DSM_RULE_SOURCES, FINAL_TOOL_READI-

NESS_CHECK, FORMULA_SOURCES, LOCATION_AND_SUBSTATION_DATA, ML_-PIPELINE, PHASE1_SYSTEM_VALIDATION, REAL_DATA_PHASE1_5, REAL_DATA_-PHASE1_7_BENGALURU_ML, SECURITY_PLAN, SOURCE_REGISTRY, TOOLING_-AND_BROWSER_SETUP_REPORT, formulas, and substation_data_quality_report, tariff_and_dsm_source_verification.
Not present in repository: `docs/REAL_DATA_PHASE1_6.md`, `docs/limitations.md`. (Phase 1.6 was skipped; the project moved from 1.5 to 1.7.)

This report reflects the repository at commit state of 6 July 2026. Metrics are quoted from model cards and manifests; endpoints from the live route table; test, lint, and build results from the commands in Table 9. Unverified items are marked accordingly and not asserted as complete.

References

[1] Fastapi web framework. `https://fastapi.tiangolo.com/`.

[2] Kptcl state load despatch centre (karnataka). `https://kptclsldc.in/`. Real-time load despatch/SCADA/energy accounting portal; no clean public API. Not wired.

[3] Nasa power (prediction of worldwide energy resources). `https://power.larc.nasa.gov/`. Solar/meteorological point API (daily and hourly). Accessed 2026-07.

[4] Next.js react framework (v14). `https://nextjs.org/`.

[5] Open-meteo free weather api (forecast and historical archive/era5). `https://open-meteo.com/en/docs`. CC BY 4.0; historical archive endpoint archive-api.open-meteo.com. Accessed 2026-07.

[6] Openstreetmap overpass api (`power=substation`). `https://overpass-api.de/api/interpreter`. Data (C) OpenStreetMap contributors, ODbL 1.0. Accessed 2026-07.

[7] pvlib-python: photovoltaic modelling (erbs, ineichen, faiman, pvwatts). `https://pvlib-python.readthedocs.io/`. BSD-3-Clause.

[8] scikit-learn: Machine learning in python. `https://scikit-learn.org/`. RandomForest, HistGradientBoosting estimators used.

[9] Central Electricity Regulatory Commission, India. Cerc (deviation settlement mechanism and related matters) regulations, 2024. `https://cercind.gov.in/`. Framework verified; exact "X"/rates set by CERC order (pending).

[10] dronio. Solar radiation prediction (nasa hi-seas). `https://www.kaggle.com/datasets/dronio/SolarEnergy`. Hawaii HI-SEAS; irradiance target. Registered as REAL_NON_-LOCAL / pretraining only.

[11] Karnataka Electricity Regulatory Commission. Kerc forecasting, scheduling and deviation settlement mechanism framework (karnataka). `https://kerc.karnataka.gov.in/`. Framework verified; exact current tariff/slab order pending.

[12] smarthkaushal. National-level electricity load curve data (india). `https://www.kaggle.com/datasets/smarthkaushal/energy-demand-profile`. Kaggle dataset; "Hourly Demand Met (MW)", India national aggregate.